

`useReducer` in React

`useReducer` is a React Hook used for **complex state management**.

It is an alternative to `useState`.

You usually use `useReducer` when:

- State logic is complex
- Multiple state values depend on each other
- State updates involve many conditions
- You want Redux-like state management inside components

Syntax

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Understanding the Terms

1. `state`

Current state data.

Example:

```
{  
  count: 0  
}
```

2. `dispatch`

Function used to send actions to reducer.

Example:

```
dispatch({ type: "INCREMENT" })
```

3. `reducer`

A function that decides how state changes.

Syntax:

```
function reducer(state, action) {  
  // logic  
}
```

It takes:

Parameter	Meaning
state	Current state
action	What operation to perform

Returns:

```
newState
```

4. `initialState`

Starting value of state.

Example:

```
const initialState = { count: 0 };
```

Basic Flow of `useReducer`

```
dispatch(action)  
  ↓  
reducer(state, action)  
  ↓  
returns new state  
  ↓  
UI updates
```

Example 1 — Counter App

Step 1: Import Hook

```
import React, { useReducer } from 'react';
```

Step 2: Create Initial State

```
const initialState = {  
  count: 0  
};
```

Step 3: Create Reducer Function

```
function reducer(state, action) {  
  switch(action.type) {  
    case "INCREMENT":  
      return {  
        count: state.count + 1  
      };  
    case "DECREMENT":  
      return {  
        count: state.count - 1  
      };  
    case "RESET":  
      return {  
        count: 0  
      };  
    default:  
      return state;  
  }  
}
```

Step 4: Use `useReducer`

```
const [state, dispatch] = useReducer(reducer, initialState);
```

Complete Example

```
import React, { useReducer } from 'react';

const initialState = {
  count: 0
};

function reducer(state, action) {

  switch(action.type) {

    case "INCREMENT":
      return {
        count: state.count + 1
      };

    case "DECREMENT":
      return {
        count: state.count - 1
      };

    case "RESET":
      return {
        count: 0
      };

    default:
      return state;
  }
}

function Counter() {

  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <div>

      <h1>Count: {state.count}</h1>

      <button onClick={() => dispatch({ type: "INCREMENT" })}>
        Increment
      </button>

      <button onClick={() => dispatch({ type: "DECREMENT" })}>
        Decrement
      </button>

    </div>
  );
}
```

```
        <button onClick={() => dispatch({ type: "RESET" })}>
          Reset
        </button>

      </div>
    );
  }

export default Counter;
```

How This Works

Initial Render

```
state = { count: 0 }
```

UI shows:

```
Count: 0
```

When Increment Button Clicks

```
dispatch({ type: "INCREMENT" })
```

Reducer receives:

```
state = { count: 0 }
```

```
action = { type: "INCREMENT" }
```

Reducer returns:

```
{ count: 1 }
```

React updates UI.

Why Use `useReducer` Instead of `useState`?

Using `useState`

```
const [count, setCount] = useState(0);
```

Good for simple state.

Problem with Complex State

Example:

```
{
  users: [],
  loading: false,
  error: null
}
```

Managing this with many `useState` hooks becomes messy.

`useReducer` Solves This

All logic stays inside one reducer.

Cleaner and scalable.

Example 2 — Todo App

Complete Example

```
import React, { useReducer, useState } from 'react';

const initialState = {
  todos: []
};

function reducer(state, action) {

  switch(action.type) {

    case "ADD_TODO":
      return {
        todos: [...state.todos, action.payload]
      };

    case "DELETE_TODO":
      return {
        todos: state.todos.filter(
          (todo, index) => index !== action.payload
        )
      };

    default:
      return state;
  }
}

function TodoApp() {

  const [text, setText] = useState("");

  const [state, dispatch] = useReducer(reducer, initialState);

  const addTodo = () => {

    dispatch({
      type: "ADD_TODO",
      payload: text
    });

    setText("");
  };

  return (
    <div>

      <input
```

```

        type="text"
        value={text}
        onChange={ (e) => setText(e.target.value)}
      />

      <button onClick={addTodo}>
        Add
      </button>

      <ul>

        {state.todos.map((todo, index) => (

          <li key={index}>

            {todo}

            <button
              onClick={() =>
                dispatch({
                  type: "DELETE_TODO",
                  payload: index
                })
              }
            >
              Delete
            </button>

          </li>

        ) )}

      </ul>

    </div>
  );
}

export default TodoApp;

```

Understanding `action.payload`

Actions can carry data.

Example:

```
dispatch({
  type: "ADD_TODO",
  payload: "Learn React"
});
```

Here:

Property	Meaning
type	Action name
payload	Data sent

Real-World Example

`useReducer` is useful in:

- Authentication forms
 - Shopping cart
 - Multi-step forms
 - Todo apps
 - Dashboard filters
 - API state handling
 - Game logic
-

Example 3 — API State Management

```
{
  loading: true,
  data: [],
  error: null
}
```

Actions:

```
FETCH_START
FETCH_SUCCESS
FETCH_ERROR
```

Very common pattern.

Reducer Example for API

```
function reducer(state, action) {

  switch(action.type) {

    case "FETCH_START":
      return {
        ...state,
        loading: true
      };

    case "FETCH_SUCCESS":
      return {
        loading: false,
        data: action.payload,
        error: null
      };

    case "FETCH_ERROR":
      return {
        loading: false,
        data: [],
        error: action.payload
      };

    default:
      return state;
  }
}
```

Important Concepts

1. State Must Be Immutable

Wrong:

```
state.count++;  
return state;
```

Correct:

```
return {  
  count: state.count + 1  
};
```

Never directly modify state.

2. Reducer Must Be Pure

Reducer should:

- ☐ Return same output for same input
 - ☐ Not call APIs
 - ☐ Not use timers
 - ☐ Not modify external variables
-

3. Multiple State Values

Example:

```
const initialState = {  
  name: "",  
  email: "",  
  age: 0  
};
```

Perfect use case for `useReducer`.

`useState` **VS** `useReducer`

Feature	<code>useState</code>	<code>useReducer</code>
Simple state	Excellent	Good
Complex state	Hard	Excellent
Multiple related states	Hard	Excellent
Scalable	Medium	High
Redux-like pattern	No	Yes

Best Practices

Use Constants for Action Types

```
const INCREMENT = "INCREMENT";
```

Split Reducers for Large Apps

Large reducers become hard to manage.

Create separate reducer files.

Combine with Context API

Powerful combination:

```
Context API + useReducer
```

Used for global state management.

Common Mistakes

Mistake 1

Mutating state directly.

☐ Wrong

```
state.count++;
```

Mistake 2

Forgetting default case.

☐ Wrong

```
switch(action.type) {  
}
```

Always add:

```
default:  
  return state;
```

Mental Model

Think of `useReducer` like:

Current State + Action → New State

Example:

```
{ count: 0 } + INCREMENT  
↓  
{ count: 1 }
```

Simple Analogy

Imagine ATM machine.

Current Balance

₹1000

Action

Withdraw ₹200

Reducer Logic

1000 - 200

New State

₹800

That is exactly how `useReducer` works.

When NOT to Use `useReducer`

Avoid for simple states like:

```
const [name, setName] = useState("");
```

`useState` is easier there.

Final Summary

`useReducer`:

- ☐ Centralizes state logic
 - ☐ Handles complex state easily
 - ☐ Makes code scalable
 - ☐ Similar to Redux pattern
 - ☐ Great for large React applications
-

Most Important Interview Question

Difference Between `useState` and `useReducer`

`useState`

Used for simple independent states.

`useReducer`

Used for complex state logic where multiple updates depend on actions and previous state.